UNIT–I: BASICS OF PROGRAMMING

Approaches to Problem Solving

In computer programming, **problem solving** is the systematic process of understanding a problem, designing a suitable solution, converting the solution into a program, and checking whether the program produces the correct result.

A computer cannot solve a problem automatically unless the programmer gives it a **clear and logically ordered set of instructions**.

Therefore, programming is not only writing code. The programmer must first:

Understand the problem → Analyze it → Design the solution → Write the program → Test it → Correct errors → Maintain it

Definition

Problem solving is a systematic process of analyzing a problem, designing an algorithm for its solution, implementing the algorithm using a programming language, and testing the program to obtain the desired result.

Steps in Problem Solving

A systematic problem-solving process generally consists of the following stages:

1. Problem Definition

The first step is to understand **what exactly has to be solved**.

The programmer should identify:

- What is the problem?
- What data is given?
- What output is required?
- What conditions or restrictions exist?

Example

Suppose the problem is:

"Find the largest of three numbers."

Here:

Input: Three numbers `a`, `b`, and `c`
Output: Largest number
Condition: All three numbers must be compared.

A clearly defined problem makes the remaining stages easier.

2. Problem Analysis

After defining the problem, it is analyzed in detail.

The programmer determines:

- Required inputs
- Expected outputs
- Processing required
- Mathematical formulas, if any
- Conditions and constraints
- Data types required

Example: Simple Interest

Problem:

Calculate simple interest.

We know:

$SI=(P\times R\times T) / 100$

Where:

- `P` = Principal amount
- `R` = Rate of interest
- `T` = Time
- `SI` = Simple Interest

Therefore:

Input: P, R, T
Processing: `(P × R × T) / 100`
Output: SI

This is called **IPO analysis**.

IPO Model

Input → Processing → Output

It is one of the simplest techniques for analyzing a programming problem.

3. Solution Design

After analyzing the problem, a solution is designed.

Normally, the solution is represented using:

- Algorithm
- Flowchart
- Pseudocode

At this stage, we concentrate on **logic**, not on the syntax of a particular programming language.

Example Algorithm: Find Larger of Two Numbers

Step 1: Start
Step 2: Read `A` and `B`
Step 3: If `A > B`, display `A`
Step 4: Otherwise, display `B`
Step 5: Stop

4. Coding or Implementation

In this step, the designed solution is converted into a program using a programming language such as **C**.

Example:

```
#include <stdio.h>

int main()
{
    int a, b;

    printf("Enter two numbers: ");
    scanf("%d%d", &a, &b);

    if(a > b)
        printf("%d is larger", a);
    else
        printf("%d is larger", b);

    return 0;
}
```

The algorithm describes **what to do**, while the program describes the solution in a language that can ultimately be executed by a computer.

5. Compilation

The source program written in C must be translated into machine-understandable form.

A **compiler** checks the source code and translates it into object/machine code.

For example, an error such as:

```
printf("Hello")
```

may produce a compilation error because the semicolon is missing.

Correct statement:

```
printf("Hello");
```

Such errors are generally called **syntax errors**.

6. Testing and Debugging

After coding, the program must be tested with different input values.

Testing means executing a program using suitable test data to determine whether it produces the expected output.

Debugging means finding and removing errors, or **bugs**, from the program.

Major Types of Errors

1. Syntax Error

Violation of programming-language rules.

Example:

```
int a
printf("%d", a);
```

The semicolon after `int a` is missing.

2. Runtime Error

An error that occurs while the program is running.

Example:

```
c = a / b;
```

If `b = 0`, division by zero can cause a runtime problem.

3. Logical Error

The program executes but gives an incorrect result.

Example:

```
area = 2 * 3.14 * r;
```

If we wanted the area of a circle, this logic is incorrect.

Correct formula:

```
area = 3.14 * r * r;
```

Logical errors can be particularly difficult to detect because the program may compile and run successfully.

7. Documentation

Documentation means preparing information about the program so that users and programmers can understand how it works.

It may include:

- Problem description
- Algorithm
- Flowchart
- Program comments
- Input/output description
- Function descriptions
- User instructions

Example:

```
/* Program to calculate simple interest */
```

Good documentation makes a program easier to understand, modify, debug, and maintain.

8. Maintenance

Software may require modification even after it has been successfully developed.

Maintenance may be required to:

- Correct errors
- Improve performance
- Add new features
- Modify existing functionality
- Adapt the program to a new environment

Thus, program development does not necessarily end after testing.

Problem-Solving Cycle

```
Problem Definition
       ↓
Problem Analysis
       ↓
Solution Design
       ↓
Algorithm / Flowchart
       ↓
Coding
       ↓
Compilation
       ↓
Testing & Debugging
       ↓
Documentation
       ↓
Maintenance
```

Major Approaches to Problem Solving

Different approaches can be used to design solutions. Two important approaches in programming are:

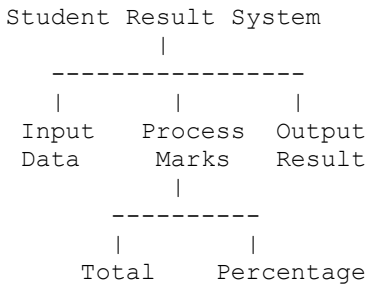
1. Top-Down Approach

In the **top-down approach**, a large problem is divided into smaller and simpler subproblems.

Each subproblem is further divided until it becomes easy to solve.

It is also known as **stepwise refinement**.

For example, consider a **Student Result System**.



Instead of developing the entire system at once, we divide it into smaller modules.

Advantages

- Easy to understand
- Reduces complexity
- Easy debugging
- Easy testing
- Supports modular programming
- Easy maintenance
- Individual modules can be developed separately

Example in C

```
int main()
{
    input();
    calculate();
    display();
}
```

The main problem is divided into functions such as `input()`, `calculate()`, and `display()`.

2. Bottom-Up Approach

The **bottom-up approach** starts with small, basic components and combines them to construct a larger system.

Instead of starting with the complete problem and dividing it, we first develop reusable lower-level modules.

For example:

```
Input Module
      +
Calculation Module
      +
Display Module
      ↓
Student Result System
```

Advantages

- Promotes code reuse

- Individual modules can be tested separately
- Useful for large software systems
- Existing components can be reused
- Reduces repeated development

Top-Down vs Bottom-Up Approach

Basis	Top-Down Approach	Bottom-Up Approach
Starting point	Complete problem	Small components
Process	Large problem is divided	Small modules are combined
Direction	General to specific	Specific to general
Main idea	Decomposition	Composition
Design focus	Overall system first	Components first
Testing	Higher-level structure is considered first	Individual components are tested first
Example	Divide result system into modules	Combine input, calculation and output modules

3. Structured Approach to Problem Solving

Structured programming is an approach in which a program is developed using well-defined control structures and is divided into manageable modules.

The three fundamental control structures are:

1. Sequence

Instructions execute one after another.

```
scanf("%d%d", &a, &b);
sum = a + b;
printf("%d", sum);
```

Flow:

```
Input
  ↓
Calculation
  ↓
Output
```

2. Selection

One of different alternatives is selected depending on a condition.

Examples:

```
if
if-else
switch
```

Example:

```
if(marks >= 40)
    printf("Pass");
else
    printf("Fail");
```

3. Iteration

A set of instructions is repeated.

Examples:

```
for
while
do-while
```

Example:

```
for(int i = 1; i <= 10; i++)
    printf("%d ", i);
```

Thus:

Structured Programming = Sequence + Selection + Iteration + Modular Design

Use of High-Level Programming Language for Systematic Development of Programs

What is a Programming Language?

A **programming language** is a formal language used to write instructions that direct a computer to perform specific operations.

Programming languages can broadly be classified as:

```
Programming Languages
        |
  -----------------
  |               |
Low-Level       High-Level
Languages       Languages
  |               |
Machine         C
Assembly        C++
                Java
                Python
```

High-Level Programming Language

A **high-level programming language (HLL)** is a programming language whose syntax is closer to human-readable or mathematical notation than to machine instructions.

Examples include:

- C , C++
- Java
- Python

Exam Definition

A high-level language is a programming language designed to allow programmers to express algorithms using human-readable constructs while hiding many hardware-specific details.

For example, in C:

```
sum = a + b;
```

This is much easier for a programmer to understand than a sequence of machine instructions.

Why Do We Need High-Level Languages?

Computers directly operate on machine-level instructions, but writing large programs directly in machine code is difficult.

High-level languages provide **abstraction** between the programmer and the hardware.

Consider the task:

Add two numbers and display their sum.

```
#include <stdio.h>

int main()
{
    int a, b, sum;

    scanf("%d%d", &a, &b);
    sum = a + b;

    printf("%d", sum);

    return 0;
}
```

The programmer does not need to manually manage the processor instructions required to perform the addition and display operation.

Features of High-Level Languages

1. Easy to Understand

Programs are relatively readable because high-level languages use meaningful constructs.

Example:

```
total = marks1 + marks2;
```

2. Easy to Write

The programmer can express complex operations using comparatively fewer instructions.

3. Machine Independence

High-level programs are generally less dependent on a particular processor architecture than machine or assembly language.

However, the program must be translated appropriately for the target system.

4. Easy Debugging

Errors are easier to identify because source code is human-readable.

5. Easy Maintenance

Programs can be modified and updated relatively easily.

6. Supports Structured Programming

Languages such as C provide:

- Functions
- Loops
- Conditional statements
- Blocks
- Arrays and other data structures

These help organize programs systematically.

7. Modularity

A large program can be divided into smaller functions or modules.

Example:

```
readData();
calculateResult();
displayResult();
```

8. Portability

Properly written standard C programs can often be compiled on different platforms with little or no modification.

9. Reusability

Functions and modules can be reused in other parts of a program or in other programs.

Role of High-Level Language in Systematic Program Development

A high-level language allows the programmer to convert a designed solution into a **structured, readable and maintainable program**.

The systematic development can be represented as:

```
Problem
   ↓
Requirement Analysis
   ↓
Algorithm
   ↓
Flowchart / Pseudocode
   ↓
High-Level Language Program
   ↓
Compiler
   ↓
Executable Program
   ↓
```

```
Testing
   ↓
Result
```

Example: Systematic Development of a Program

Problem: Write a program to determine whether a number is even or odd.

Step 1: Identify Input

Input = An integer `n`

Step 2: Identify Output

Output = `"Even"` or `"Odd"`

Step 3: Determine Logic

A number is even if:

n%2==0 → Number is Even

Step 4: Write Algorithm

```
START
Read n
If n % 2 == 0
    Print "Even"
Else
    Print "Odd"
STOP
```

Step 5: Convert Algorithm into C Program

```
#include <stdio.h>

int main()
{
    int n;

    printf("Enter a number: ");
    scanf("%d", &n);

    if(n % 2 == 0)
        printf("Even");
    else
        printf("Odd");

    return 0;
}
```

Step 6: Test the Program

For input: `8`

Expected output: `Even`

For input: `7`

Expected output: `Odd`

This demonstrates **systematic program development**: the programmer does not start coding randomly. The problem is analyzed, logic is designed, code is written, and then the solution is tested.

Advantages of High-Level Languages

For an AKTU theory question such as **"Explain the advantages of high-level programming languages"**, write points such as:

1. Readability – Programs are comparatively easy to read and understand.
2. Simplicity – Complex machine operations can be represented using simpler statements.
3. Portability – Programs can often be moved between different systems.
4. Easy debugging – Errors can be located and corrected more easily.
5. Easy maintenance – Existing programs can be modified efficiently.
6. Faster development – Less code is generally required than low-level programming.
7. Modularity – Large programs can be divided into functions/modules.
8. Reusability – Existing functions and modules can be reused.
9. Abstraction – Hardware-level details are largely hidden from the programmer.
10. Structured development – Algorithms can be systematically converted into programs.

Limitations of High-Level Languages

High-level languages also have some limitations:

- They require a **translator**, such as a compiler or interpreter.
- The programmer has less direct control over hardware than with low-level languages.
- Depending on the language and implementation, translated programs may have some performance or memory overhead compared with carefully optimized low-level code.
- Hardware-specific operations may be more difficult to express directly.

C is somewhat special because it provides many high-level programming features while also allowing relatively low-level memory manipulation through pointers and related facilities.

High-Level Language vs Low-Level Language

High-Level Language	Low-Level Language
Easier for humans to understand	More difficult for humans to understand
More machine-independent	More machine-dependent
Easier to write	Difficult to write
Easier to debug	Debugging is comparatively difficult
Faster program development	Development usually takes more time
Hardware details are largely abstracted	Greater direct hardware control
Examples: C, C++, Java	Machine language, Assembly language
Requires translation	Machine code executes directly; assembly requires an assembler

Concept of Algorithm and Flowchart

1. Algorithm

An **algorithm** is a finite, ordered sequence of clear and well-defined steps used to solve a particular problem or perform a specific task.

In simple words, an algorithm describes **what to do and in what order** to obtain the desired result.

An algorithm is prepared **before writing the actual program**, so that the programmer can understand and verify the logic of the solution.

Example from Daily Life

Consider the task of making tea:

1. Start
2. Take water in a pan.
3. Heat the water.
4. Add tea leaves.
5. Add milk and sugar.
6. Boil the mixture.
7. Filter the tea.
8. Serve the tea.
9. Stop

This is an algorithm because it provides a sequence of steps to accomplish a task.

Characteristics of an Algorithm

A good algorithm should have the following characteristics:

1. Input: An algorithm may accept **zero or more inputs**.

Example:
To calculate the area of a rectangle, `length` and `width` are inputs.

2. Output: An algorithm should produce at least one output or result.

Example:
The calculated area is the output.

3. Definiteness: Every step must be **clear, precise and unambiguous**.

"Perform some calculation."

"Calculate `Area = Length × Width`."

4. Finiteness: An algorithm must terminate after a **finite number of steps**.

It should not continue indefinitely.

5. Effectiveness

Every step should be **simple, practical and executable** within a reasonable amount of time.

6. Correctness: The algorithm should produce the **correct result** for valid input.

7. Generality

A good algorithm should solve a **class of similar problems**, rather than only one specific input.

Advantages of Algorithms

Algorithms provide several benefits during program development:

- Help in understanding the problem
- Provide a step-by-step solution
- Make program logic easier to understand
- Help detect logical errors before coding
- Make coding easier
- Simplify testing and debugging
- Improve communication between programmers
- Make documentation easier
- Can be converted into different programming languages

Limitations of Algorithms

Although algorithms are useful, they also have some limitations:

- Complex problems may require very long algorithms.
- Writing detailed algorithms can be time-consuming.
- Algorithms do not provide a visual representation of logic.
- For highly complex problems, a flowchart may provide better understanding.

Problem: Find the largest of two numbers

Algorithm:

1. Start
2. Read two numbers `A` and `B`.
3. Compare `A` and `B`.
4. If `A > B`, display `A` as the largest number.
5. Otherwise, display `B` as the largest number.
6. Stop

Representation

Input → Compare → Decision → Output

```
A, B → A > B? → Yes → A is largest
              → No → B is largest
```

FLOW CHARTING

A **flowchart** is a graphical representation of the steps involved in solving a problem or performing a process.

Instead of writing all steps in text, we represent them using standard symbols connected by arrows.

A flowchart is a diagrammatic representation of an algorithm or process using standard graphical symbols.

Why are Flowcharts Used?

Flowcharts help programmers:

- Understand a problem
- Visualize program logic
- Identify errors
- Communicate the solution
- Explain a program to others
- Plan the program before coding
- Document the logic

Standard Flowchart Symbols

Students should remember these symbols carefully because **flowchart symbol questions are common in examinations.**

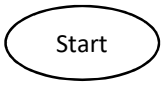
Symbol	Name	Purpose
Oval	Terminal	Start/Stop
Rectangle	Process	Calculation/processing
Parallelogram	Input/Output	Input or output
Diamond	Decision	Condition/decision
Arrow	Flow line	Direction of flow
Circle	Connector	Connects flowchart sections

Terminal Symbol

The terminal symbol represents:

- Start
- End
- Stop

Example:



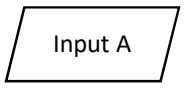
Usually represented by an **oval**.

Input / Output Symbol

The parallelogram is used for:

- Taking input
- Displaying output

Examples:



Process Symbol

A rectangle represents a processing operation.

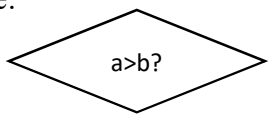
Example:

```
┌──────────────────┐
│ Sum = A + B      │
└──────────────────┘
```

Decision Symbol

A diamond represents a decision or condition.

Example:



A decision normally has two or more possible paths.

Flow Line

An arrow shows the direction in which the process moves.

Example:

```
START -> INPUT -> PROCESS -> OUTPUT -> STOP
```

Basic Flowchart Rules

When creating a flowchart:

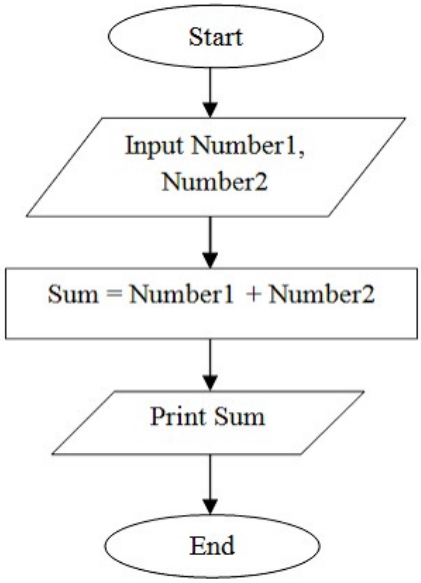
1. Start with a **Start** symbol.
2. End with a **Stop/End** symbol.
3. Use standard symbols.
4. Connect symbols using arrows.
5. Maintain a logical flow.
6. Avoid unnecessary crossing of lines.
7. Decision paths should be clearly labelled.
8. Keep the flowchart simple and readable.
9. Use meaningful statements inside symbols.
10. Follow a consistent direction wherever possible.

Flowchart for Adding Two Numbers

Algorithm

```
Step 1: Start
Step 2: Input A and B
Step 3: Calculate Sum = A + B
Step 4: Display Sum
Step 5: Stop
```

Flowchart Logic

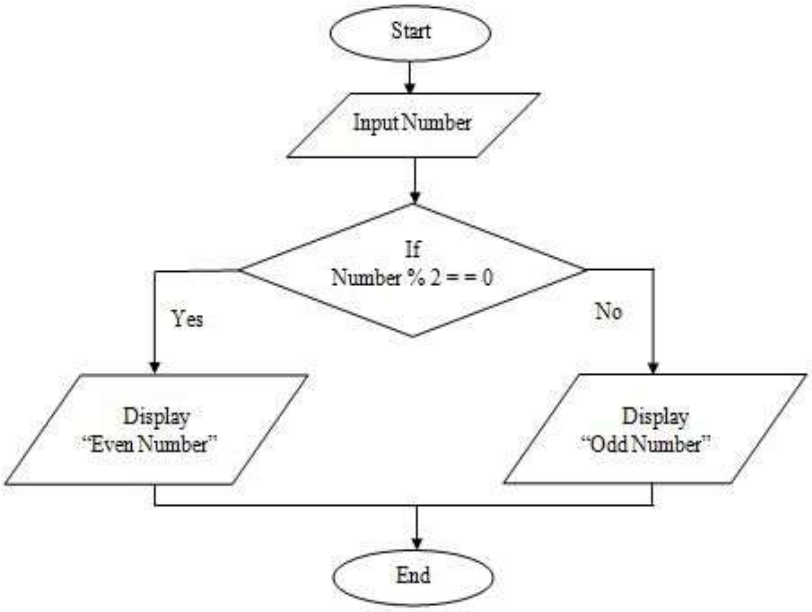


Flowchart for Finding Even or Odd Number

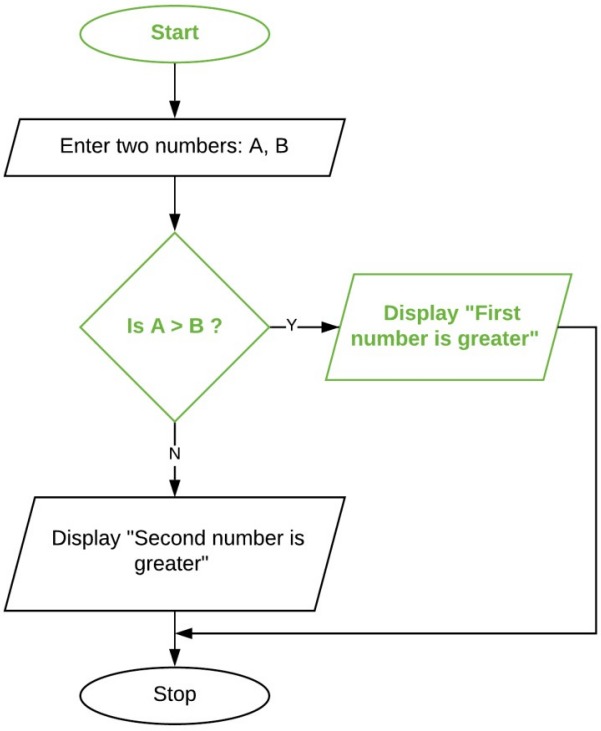
Algorithm Logic

1. Start
2. Input number
3. Check whether number is divisible by 2
4. If yes → Even
5. Otherwise → Odd
6. Stop

Flowchart



Flowchart for Finding Largest of Two Numbers



Advantages of Flowcharts

1. Easy to understand
2. Helps in problem analysis
3. Improves communication
4. Helps identify logical errors
5. Useful before coding
6. Helps in documentation
7. Makes complex processes easier to visualize

Limitations of Flowcharts

1. Time-consuming for complex programs.
2. Large flowcharts can become difficult to manage.
3. Changes may require redrawing the chart.
4. Complex programs may require very large diagrams.
5. Maintaining flowcharts can be difficult.

Rules for Drawing a Flowchart

While creating a flowchart, the following rules should generally be followed:

1. Use standard flowchart symbols.
2. Start the flowchart with a **Start** symbol.
3. End the flowchart with an **End/Stop** symbol.
4. Use arrows to show the direction of flow.
5. The flow should normally move from **top to bottom** or **left to right**.
6. Use a diamond symbol for decisions.
7. Clearly label decision branches such as **Yes/No** or **True/False**.

8. Avoid unnecessary crossing of flow lines.
9. Keep the flowchart simple and easy to understand.
10. Each process should be clearly defined.

Algorithm vs Flowchart

Algorithm	Flowchart
Written representation of solution	Graphical representation of solution
Uses statements or steps	Uses symbols and arrows
Easy to write for simple problems	Easy to understand visually
Suitable for describing detailed logic	Suitable for visualizing program flow
Can become lengthy for complex problems	Can become large and difficult for complex problems
Easier to modify	Modifications may require redrawing
Example: Step 1, Step 2, Step 3	Example: Oval, Rectangle, Diamond, Arrow

Concept and Role of Structured Programming Languages

1. Concept of Structured Programming

Structured Programming is a programming approach in which a program is divided into small, logical and manageable sections using well-defined control structures.

The main objective of structured programming is to make programs **clear, organized, reliable, easy to test, and easy to maintain**.

Instead of writing a program as a collection of uncontrolled jumps, structured programming follows a logical flow of execution.

Definition

Structured programming is a programming methodology that organizes a program using sequence, selection, and iteration control structures.

Languages such as **C, C++, Java, and Pascal** support structured programming concepts, although some of them also support other programming paradigms.

2. Basic Control Structures

Structured programming is mainly based on **three fundamental control structures**.

1. Sequence

In a sequence, statements are executed **one after another in the order in which they are written**.

Example:

```
Read A
Read B
Sum = A + B
Display Sum
```

Flow: Input → Calculation → Output

2. Selection

Selection allows the program to choose between different actions based on a condition.

Common selection statements are:

- `if`
- `if-else`
- `else-if`
- `switch`

Example:

```
if (marks >= 40)
    printf("Pass");
else
    printf("Fail");
```

Here, the program selects one of two possible paths.

3. Iteration

Iteration is used to **repeat a set of statements** until a particular condition is satisfied.

Common looping statements are:

- `for`
- `while`
- `do-while`

Example:

```
for (int i = 1; i <= 5; i++)
{
    printf("%d ", i);
}
```

The statements inside the loop are executed repeatedly.

Main Features of Structured Programming

1. Modular Design: A large program can be divided into smaller modules or functions.

Example:

```
Student Management System
          |
   ----------------
   |      |       |
Input   Process  Output
```

Each module performs a specific task.

2. Use of Control Structures: Structured programs primarily use:

Sequence + Selection + Iteration

These structures provide a clear and predictable flow of execution.

3. Avoidance of Uncontrolled `goto`:

Traditional programming often used the `goto` statement to jump from one part of a program to another.

Structured programming generally avoids unnecessary `goto` statements because they can make program logic difficult to follow.

4. Top-Down Design: A complex problem is divided into smaller sub-problems.

Large Problem → Sub-problems → Modules → Functions

This makes large programs easier to understand and develop.

5. Single Entry and Exit

A well-structured block generally has a clear point of entry and exit, making program flow easier to understand.

Role of Structured Programming Languages

Structured programming plays an important role in software development.

1. Improves Program Readability

Programs are organized into logical blocks, making the code easier for programmers to read and understand.

2. Reduces Program Complexity

A large problem can be divided into smaller and simpler modules.

For example:

Banking Application

- Login
- Account Management
- Deposit
- Withdrawal
- Transaction History

Each part can be developed and tested separately.

3. Makes Debugging Easier

Since the program is divided into logical sections, errors can be located more easily.

For example, if the calculation module produces an incorrect result, the programmer can focus on that particular module.

4. Simplifies Testing

Individual functions or modules can be tested separately before integrating them into the complete program.

5. Improves Maintainability

When requirements change, a particular module can often be modified without changing the entire program.

6. Encourages Code Reusability

Functions and modules can be reused in different parts of a program or even in other programs.

Example:

A function such as:

```
calculateTax()
```

can be called whenever tax calculation is required.

7. Improves Reliability

A well-organized program is easier to analyze, test and verify, which helps reduce programming errors.

8. Supports Team Development

Large software projects can be divided into modules and assigned to different developers.

For example:

Developer 1 → Login Module
Developer 2 → Payment Module
Developer 3 → Database Module
Developer 4 → Reporting Module

Difference Between Unstructured & Structured Approach

Unstructured Approach

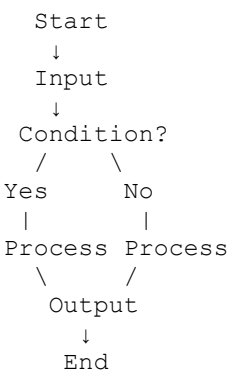
A program with many uncontrolled jumps can make the execution flow difficult to understand:

```
Start
 ↓
Statement A
 ↓
goto X
 ↓
Statement B
 ↓
goto Y
 ↓
Statement C
```

The programmer has to keep track of many jumps.

Structured Approach

The same logic can generally be organized using:



The flow is easier to understand and maintain.

Advantages of Structured Programming

- Easy to understand
- Easy to write and modify
- Reduces complexity
- Easier debugging
- Easier testing
- Better program organization
- Supports modular programming
- Improves maintainability
- Encourages code reuse
- Suitable for developing large programs

Limitations of Structured Programming

Structured programming also has some limitations:

- Large systems may become difficult to manage using only procedural techniques.
- Data and functions are not always tightly combined.
- Reusable components may require additional design.
- For very large and complex software, **object-oriented or other programming paradigms** may provide better organization.

Introduction to Programming and Program

Programming is the process of designing, writing, testing, debugging, and maintaining a set of instructions that tells a computer how to perform a particular task.

A computer cannot solve a problem on its own. It needs a sequence of instructions given by a programmer. These instructions are written using a **programming language** such as C, C++, Java, or Python.

Programming generally involves understanding a problem, developing its solution through an algorithm or flowchart, writing the program, and then testing it for correct results.

What is a Program?

A **program** is a set of instructions written in a programming language to perform a specific task or solve a particular problem.

For example, a program may be developed to:

- Calculate the salary of employees.
- Prepare students' results.
- Calculate profit and loss.
- Maintain inventory records.
- Generate bills.
- Process business transactions.

Difference Between Program and Programming

Program	Programming
It is a set of computer instructions.	It is the process of developing a program.
It is the final set of instructions given to a computer.	It includes designing, coding, testing, and debugging.
It performs a specific task.	It is used to create programs for different tasks.

Programming is the process of developing a set of instructions for a computer to solve a problem, while a program is a sequence of instructions written in a programming language to perform a specific task.

History of C Language

C is a general-purpose programming language that has played an important role in the development of modern computer programming.

The development of C is closely related to earlier programming languages such as **ALGOL, CPL, BCPL, and B**.

Development of C

ALGOL: ALGOL (Algorithmic Language) was developed around 1960 and introduced important concepts of structured programming.

CPL: Combined Programming Language (CPL) was developed in the 1960s. It was powerful but relatively complex.

BCPL: Martin Richards developed **BCPL (Basic Combined Programming Language)** in 1967. It simplified many concepts of CPL.

B Language: Ken Thompson developed the B programming language at Bell Laboratories around 1969. It was influenced by BCPL.

C Language: Dennis Ritchie developed the C programming language at **Bell Laboratories** in the early 1970s. C evolved from the B language.

C became closely associated with the development of the **UNIX operating system**. Much of UNIX was rewritten in C, demonstrating that C could be effectively used for system programming.

In 1978, **Brian Kernighan and Dennis Ritchie** published the famous book *The C Programming Language*. The version of C described in this book became widely known as **K&R C**.

Later, C was standardized by **ANSI (American National Standards Institute)**. The ANSI C standard was approved in **1989**, helping establish a common and portable form of the language.

Evolution at a Glance

ALGOL → CPL → BCPL → B → C

This sequence is useful to remember for examinations.

Important Facts About C

- **Developer:** Dennis Ritchie
- **Developed at:** Bell Laboratories
- **Period:** Early 1970s
- **Predecessor:** B Language
- **Major application:** Development of UNIX
- **Standardization:** ANSI C standard in 1989

Salient Features of C Language

C became one of the most widely used programming languages because it provides both simplicity and powerful programming facilities.

The major features of C are as follows:

1. Simple Language

C has a relatively small set of keywords and provides a clear programming structure. Therefore, its basic concepts can be learned easily.

2. Structured Programming Language

C supports **structured programming**. A large program can be divided into smaller functions or modules, making the program easier to develop, test, and maintain.

3. Middle-Level Language

C is often described as a **middle-level language** because it combines features associated with both high-level programming and relatively low-level system programming.

It provides high-level features such as functions and control statements while also allowing low-level operations through pointers and bitwise operators.

4. Portable Language

Programs written in C can generally be moved from one computer system to another with comparatively few modifications.

This property is known as **portability**.

5. Fast and Efficient

C programs can execute efficiently and provide programmers with considerable control over memory and system resources.

6. Rich Set of Operators

C provides various types of operators, including:

- Arithmetic operators
- Relational operators
- Logical operators
- Assignment operators
- Increment and decrement operators
- Bitwise operators
- Conditional operator

7. Rich Library Functions

C provides many standard library functions for operations such as input/output, mathematical calculations, string processing, and memory management.

Examples include `printf()`, `scanf()`, `strlen()`, and `sqrt()`.

8. Supports Functions

C allows a program to be divided into different **functions**. Each function can perform a specific task.

This improves modularity and makes programs easier to understand.

9. Supports Pointers

A **pointer** is a variable that stores a memory address. C provides extensive support for pointers, making it useful for memory manipulation and system programming.

10. Dynamic Memory Management

C supports dynamic memory allocation through functions such as `malloc()`, `calloc()`, `realloc()`, and `free()`.

11. Supports Recursion

C allows a function to call itself. This technique is known as **recursion**.

12. Extensible

Programmers can create their own functions and add them to existing programs. Therefore, C programs can be extended according to requirements.

13. Case Sensitive

C is a **case-sensitive language**. Uppercase and lowercase letters are treated differently.

For example, `total`, `Total`, and `TOTAL` are considered different identifiers.

Simple → Structured → Portable → Efficient → Rich Operators → Functions → Pointers

Structure of a C Program

A C program is generally organized into different sections. A proper structure makes the program easy to understand, test, debug, and maintain.

General Structure

```
/* Documentation Section */

#include <stdio.h>          // Link Section

#define VALUE 100           // Definition Section

/* Global Declaration Section */

int main()
{
    /* Local Declaration Section */

    /* Executable Statements */

    return 0;
}

/* User-defined Function Section */
```

The major sections of a C program are explained below.

1. Documentation Section

This section contains comments describing information about the program, such as its purpose, author, or functionality.

Example:

```
/* Program to calculate sum of two numbers */
```

Comments improve the readability and documentation of a program.

2. Link Section

The link section includes the required header files with the help of the `#include` directive.

Example:

```
#include <stdio.h>
```

Here, `stdio.h` provides declarations for standard input/output functions such as `printf()` and `scanf()`.

3. Definition Section

This section is used to define symbolic constants or macros, commonly with `#define`.

Example:

```
#define PI 3.14159
```

4. Global Declaration Section

Variables and function prototypes that need to be accessible to multiple functions may be declared outside the functions.

Example:

```
int total;
```

Such a variable declared outside functions is generally called a **global variable**.

5. `main()` Function Section

The `main()` function is the entry point of a hosted C program. Program execution starts from `main()`.

Example:

```
int main()
{
    /* statements */
    return 0;
}
```

The `main()` function normally contains declarations and executable statements.

6. Declaration Part

Variables required inside a function are declared here.

Example:

```
int a, b, sum;
```

7. Executable Part

This part contains statements that perform the actual operations of the program.

Example:

```
sum = a + b;
printf("%d", sum);
```

8. User-Defined Function Section

A programmer may create additional functions to perform specific tasks.

Example:

```
int add(int a, int b)
{
    return a + b;
}
```

These functions help divide a large program into smaller modules.

Example of a Simple C Program

```
#include <stdio.h>

int main()
{
    int a, b, sum;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    sum = a + b;

    printf("Sum = %d\n", sum);

    return 0;
}
```

Explanation

`#include <stdio.h>` includes standard input/output declarations.

`int main()` defines the main function from where program execution begins.

`int a, b, sum;` declares three integer variables.

`printf()` displays information on the screen.

`scanf()` accepts input from the user.

`sum = a + b;` performs the addition.

`return 0;` indicates successful termination of the program to the host environment.

Compiling a C Program

A program written in C language is called a **source program** or **source code**. A computer cannot directly understand C statements because the computer understands machine language.

Therefore, the C source program must be translated into machine-understandable form. This translation is performed by a **compiler**.

Compilation is the process of translating a program written in a high-level language such as C into machine-level or object code with the help of a compiler.

A C compiler also checks the program for various errors before successful translation.

Stages of C Program Processing

A C program generally passes through the following stages:

Source Program → Preprocessing → Compilation → Assembly → Linking → Executable Program → Execution

1. Source Program

The programmer writes the C program and saves it in a file, generally with the `.c` extension.

Example: `program.c`

2. Preprocessing

The **preprocessor** processes preprocessor directives such as:

```
#include <stdio.h>
#define PI 3.14
```

It handles tasks such as header-file inclusion and macro expansion.

3. Compilation

The compiler checks the program and translates the preprocessed C code toward lower-level code. Compilation errors must be corrected before the program can be successfully built.

4. Assembly

The assembler converts assembly code into **object code**.

5. Linking

The linker combines object code with required library code and other object files to create an executable program.

6. Execution

Finally, the executable program is loaded and run. It accepts input, performs processing, and produces output according to the program instructions.

Link and Run a C Program

Linking

A C program may use predefined library functions such as `printf()` and `scanf()`. Their implementations are provided through system libraries.

The **linker** combines the required compiled program components and libraries to produce the final executable file.

Linking is the process of combining object files and required library code to create an executable program.

Running a C Program

After successful linking, the executable program can be run.

During execution:

Input → Processing → Output

For example:

```c
#include <stdio.h>

int main()
{
    int a, b, sum;

    scanf("%d %d", &a, &b);
    sum = a + b;
    printf("Sum = %d\n", sum);

    return 0;
}
```

If the input is `10` and `20`, the output will be:

```
Sum = 30
```

Compilation vs Linking

Compilation	Linking
Translates source code into lower-level/object code as part of the build process.	Combines object files and libraries.
Mainly performed by compiler toolchain stages.	Performed by a linker.
Detects many syntax/semantic compilation errors.	Can detect unresolved external references/symbols.
Occurs before final executable creation.	Produces the final executable program.

Important Exam Flow

Edit → Compile → Link → Run → Output

Character Set in C

A **character set** is the set of valid characters that can be used to write C programs.

The C character set includes letters, digits, special symbols, and whitespace characters.

1. Alphabets

C uses uppercase and lowercase English letters:

Uppercase: A to Z
Lowercase: a to z

C is **case-sensitive**, so `A` and `a` are treated as different characters.

2. Digits

C uses decimal digits:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9

3. Special Characters

C uses various special symbols, such as:

```
+ - * / % = < > ( ) { } [ ] ; , . # & ! ? : ' " _
```

These symbols have different purposes in C programming.

4. White Space Characters

White spaces are used to separate program elements and improve readability.

Examples include:

- Blank space
- Tab
- New line

Tokens in C

A **token** is a basic lexical unit of a C program recognized by the compiler.

In simple words, tokens are the **basic building blocks of a C program**.

Consider:

```
int total = 100;
```

Its major tokens include:

`int` → Keyword
`total` → Identifier
`=` → Operator
`100` → Constant
`;` → Punctuator

Major Categories of C Tokens

Common categories include:

1. Keywords
2. Identifiers
3. Constants
4. String literals
5. Operators
6. Punctuators/special symbols

1. Keywords

A **keyword** is a reserved word that has a predefined meaning in the C language.

Keywords form part of the syntax of C and generally cannot be used as user-defined identifiers.

Examples:

```
int
char
float
```

For example:

```
int age;
```

Here, `int` is a keyword indicating an integer type.

Important Point

Traditional **ANSI C/C90 has 32 keywords**. Later C standards added more keywords, so "C has 32 keywords" should specifically be understood in the context of the classic ANSI C syllabus.

Definition

A keyword is a reserved word in C that has a predefined meaning and cannot normally be used as the name of a variable, function, or other user-defined identifier.

2. Identifiers

An **identifier** is a user-defined name given to program elements such as variables, functions, arrays, and other entities.

Examples:

```
total
marks
salary
```

In:

```
int marks;
```

`int` is a **keyword**, while `marks` is an **identifier**.

Rules for Writing Identifiers

1. An identifier can contain **letters, digits, and underscore (_)**.
2. It must not begin with a digit.
3. Spaces are not allowed.
4. C keywords cannot be used as identifiers.
5. C identifiers are case-sensitive.
6. Special symbols such as `@`, `%`, `#`, and - cannot form part of an ordinary identifier.

Valid Identifiers

```
marks
total_marks
salary1
studentName
_amount
```

Invalid Identifiers

```
2marks       → starts with a digit
total marks  → contains space
float        → keyword
total-marks  → contains hyphen
```

3. Constants

A **constant** is a value that remains fixed and does not change during its use as a literal value in the program.

Examples:

```
10, 25.5
'A', "Hello"
```

Types of Constants

1. Integer Constants

Whole numbers without a fractional part.

Examples: `10, 250, -50`

2. Floating-Point Constants

Numbers containing a fractional part or written in floating/exponential notation.

Examples: `10.5, 3.14, -25.75`

3. Character Constants

A single character written within single quotation marks.

Examples: `'A', 'B', '5'`

4. String Literals

A sequence of characters enclosed in double quotation marks.

Examples: `"Hello", "Computer Application", "MJPRU"`

4. Variables

A **variable** is a named object/storage location used to hold a value that may change during program execution.

For example:

```
int age = 20;
```

Here:

`int` → Data type
`age` → Variable/identifier
`20` → Initial value

The value may later be changed:

```
age = 21;
```

Variable Declaration

A variable should be declared with an appropriate data type before normal use.

General syntax:

```
data_type variable_name;
```

Examples:

```
int marks;
float salary;
char grade;
```

Variable Initialization

Assigning an initial value when a variable is declared is called **initialization**.

Example:

```
int marks = 80;
float price = 25.50;
char grade = 'A';
```

Rules for Variable Names

Variable names follow identifier rules:

- They should begin with a letter or underscore, not a digit.
- They may contain letters, digits, and underscores.
- Spaces are not allowed.
- Keywords cannot be used as variable names.
- C is case-sensitive.

For example, `salary` and `Salary` are different identifiers.

Constant vs Variable

Constant	Variable
Represents a fixed value.	Holds a value that can be changed.
Example: `100`, `3.14`, `'A'`	Example: `marks`, `salary`, `age`
A literal value itself does not change.	Its stored value may change during execution.

Instructions in C

An **instruction** or statement tells the computer to perform a particular operation.

A C program consists of declarations, expressions, statements, function calls, and control structures that collectively specify the required processing.

For introductory examination purposes, C instructions are often discussed under the following categories.

1. Type Declaration Instructions

These are used to declare variables and specify their data types.

Examples:

```
int a;
float salary;
char grade;
```

They tell the compiler what type of data a variable will hold.

2. Arithmetic Instructions

Arithmetic instructions perform mathematical calculations.

Example:

```
sum = a + b;
profit = selling_price - cost_price;
average = total / 5;
```

Common arithmetic operators are:

`+` Addition
`-` Subtraction
`*` Multiplication
`/` Division
`%` Remainder

3. Control Instructions

Control instructions determine or alter the normal flow of program execution.

Major control mechanisms include:

Decision/Selection:

```
if
if-else
switch
```

Looping/Iteration:

```
For, while, do-while
```

Jump statements:

```
break, continue, goto, return
```

For example:

```
if (marks >= 40)
{
    printf("Pass");
}
else
{
    printf("Fail");
}
```

Data Types in C

A **data type** specifies the type of data that a variable can store and the kinds of operations that can be performed on that data.

When a variable is declared in C, its data type must normally be specified.

For example:

```
int age;
float salary;
char grade;
```

Here, `int`, `float`, and `char` are data types.

Definition: A data type is a specification that tells the compiler what kind of value a variable can store and how that value should be interpreted.

Classification of Data Types

C data types can broadly be classified as:

Data Types in C

→ Basic/Primary Data Types
→ Derived Data Types
→ User-Defined Data Types
→ `void` Type

1. Basic or Primary Data Types

The fundamental data types commonly used in C include:

(a) `int`

The `int` data type is used to store **integer or whole-number values**.

Example:

```
int age = 20;
int marks = 85;
```

Values such as `10`, `-25`, and `500` can be represented using integer types when they are within the type's supported range.

(b) `char`

The `char` data type is used to store a character value.

Example:

```
char grade = 'A';
```

A character constant is written within **single quotation marks**.

(c) `float`

The `float` data type is used to store single-precision floating-point numbers.

Example:

```
float price = 45.50f;
```

(d) `double`

The `double` data type is used for floating-point values with greater precision than `float` on typical C implementations.

Example:

```
double amount = 12500.75;
```

Common Data Types at a Glance

Data Type	Used For	Example
`char`	Character/small integer type	`'A'`
`int`	Integer	`100`
`float`	Floating-point number	`25.5`
`double`	Higher-precision floating-point number	`125.5678`

Important: Exact size and range of C data types are **implementation-dependent**. For example, an `int` is commonly 4 bytes on modern systems, but the C language standard does not require it to always be 4 bytes.

Type Modifiers in C

C provides type modifiers that can modify the range or representation of certain basic data types.

The important modifiers are:

- `signed` , `unsigned`
- `short`, `long`

Examples:

```
short int a;
long int population;
unsigned int count;
long double value;
```

Signed and Unsigned

A **signed** integer type can represent negative as well as non-negative values.

An **unsigned** integer type represents only non-negative values and therefore provides a larger non-negative range for the corresponding integer width.

2. Derived Data Types

Derived data types are constructed using fundamental data types.

Important examples include:

Array

An array stores multiple elements of the same type.

```
int marks[5];
```

Pointer

A pointer stores the address of another object/function as appropriate to its pointer type.

```
int *ptr;
```

Function Type

Functions are used to perform specific tasks and have function types based on their return and parameter types.

```
int add(int a, int b);
```

3. User-Defined Data Types

C provides mechanisms through which programmers can define or name more complex data types.

Important examples include:

Structure

A structure groups related data items, potentially of different types.

```
struct Student
{
    int roll;
    char grade;
};
```

Union

A union allows its members to share the same storage region.

Enumeration

An `enum` defines a set of named integer constants.

`typedef`

`typedef` creates an alternative name for an existing type.

The `void` Type

`void` represents the absence of a value/type in specific C contexts.

For example:

```
void display(void)
{
    printf("Hello");
}
```

Here, the function does not return a value, and `void` in the parameter list indicates that it takes no parameters.

Standard Input and Output in C

Input means supplying data to a program for processing.

Output means presenting the result produced by the program.

For example:

Input → Processing → Output

```
10, 20 → Addition → 30
```

C provides standard input/output facilities through the standard I/O library. The header file commonly used is:

```
#include <stdio.h>
```

Two of the most important introductory functions are:

- `printf()` – formatted output
- `scanf()` – formatted input

`printf()` Function

The `printf()` function is used to display **formatted output** on the standard output stream, usually the screen/terminal.

Syntax

```
printf("format string", arguments);
```

Example:

```
int age = 20;
printf("Age = %d\n", age);
```

Output:

```
Age = 20
```

Common Format Specifiers

Format Specifier	Common Use
`%d`	Signed decimal `int`
`%u`	Unsigned decimal `unsigned int`
`%f`	Floating-point output

`%c`	Character
`%s`	String
`%lf`	In `scanf()`, reads a `double`

For example:

```
printf("%d", age);
printf("%c", grade);
printf("%f", salary);
```

Exam caution: In `printf()`, `%f` is used for floating-point output because a `float` argument is promoted to `double` in a variadic call. In `scanf()`, `%f` expects a `float *`, while `%lf` expects a `double *`.

24. scanf() Function

The `scanf()` function is used to read **formatted input** from standard input, usually the keyboard.

Syntax

```
scanf("format string", &variable);
```

Example:

```
int age;

printf("Enter age: ");
scanf("%d", &age);
```

The `&` operator supplies the address of the variable where the input value should be stored.

For multiple values:

```
int a, b;

scanf("%d %d", &a, &b);
```

Example of Input and Output

```
#include <stdio.h>

int main(void)
{
    int a, b, sum;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    sum = a + b;

    printf("Sum = %d\n", sum);

    return 0;
}
```

If the user enters:

```
10 20
```

Output:

```
Sum = 30
```

Operators in C

An **operator** is a symbol that instructs the program to perform a particular operation on one or more operands.

For example:

```
c = a + b;
```

Here:

- `a` and `b` are operands.
- `+` is an arithmetic operator.
- `=` is an assignment operator.

Definition: An operator is a symbol used to perform an operation on one or more operands.

Major Types of Operators

The major operator categories include:

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. Assignment Operators
5. Increment and Decrement Operators
6. Conditional Operator
7. Bitwise Operators
8. Special/Miscellaneous Operators

1. Arithmetic Operators

Arithmetic operators perform mathematical calculations.

Operator	Meaning	Example
`+`	Addition	`a + b`
`-`	Subtraction	`a - b`
`*`	Multiplication	`a * b`
`/`	Division	`a / b`
`%`	Remainder	`a % b`

Example:

```
int a = 10, b = 3;

printf("%d\n", a + b);   /* 13 */
printf("%d\n", a - b);   /* 7 */
printf("%d\n", a * b);   /* 30 */
printf("%d\n", a / b);   /* 3 */
printf("%d\n", a % b);   /* 1 */
```

Important Point: Integer Division

If both operands are integers, integer division discards the fractional part.

For example:

```
5 / 2
```

gives:

```
2
```

not `2.5`.

2. Relational Operators

Relational operators compare two values.

Operator	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Example:

```
if (marks >= 40)
{
    printf("Pass");
}
```

A relational expression evaluates to **1 when true and 0 when false** in C.

3. Logical Operators

Logical operators are commonly used to combine or negate conditions.

Operator	Meaning
&&	Logical AND
`	Logical OR
!	Logical NOT

AND `&&`

True when both conditions are true.

```
if (age >= 18 && age <= 60)
```

OR `||`

True when at least one condition is true.

```
if (grade == 'A' || grade == 'B')
```

NOT `!`

Reverses the logical truth value of a condition.

```
if (!(a > b))
```

4. Assignment Operators

Assignment operators assign or update values stored in variables.

The basic assignment operator is:

```
=
```

Example:

```
a = 10;
```

C also provides compound assignment operators:

Operator	Example	Equivalent Idea
`+=`	`a += 5`	`a = a + 5`
`-=`	`a -= 5`	`a = a - 5`
`*=`	`a *= 5`	`a = a * 5`
`/=`	`a /= 5`	`a = a / 5`
`%=`	`a %= 5`	`a = a % 5`

5. Increment and Decrement Operators

These operators increase or decrease a variable by one.

Increment Operator `++`

```
a++;
```

Increases `a` by 1.

Decrement Operator `--`

```
a--;
```

Decreases `a` by 1.

They can be used in two forms:

Pre-increment: `++a`
Post-increment: `a++`

Pre-decrement: `--a`
Post-decrement: `a--`

Pre-Increment vs Post-Increment

Consider:

```
int a = 5;
int b;
```

```
b = ++a;
```

After this:

`a = 6` and `b = 6`

But:

```
int a = 5;
int b;

b = a++;
```

After this:

`b = 5` and then `a = 6`

This difference is important in exam questions.

6. Conditional Operator

C provides the conditional or **ternary operator**:

`? :`

Syntax:

```
condition ? expression1 : expression2;
```

Example:

```
max = (a > b) ? a : b;
```

If `a > b` is true, `max` gets `a`; otherwise, it gets `b`.

It is called a **ternary operator** because it operates on three operands.

7. Bitwise Operators

Bitwise operators perform operations on individual bits of integer values.

Operator	Meaning
&	Bitwise AND
`	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT
<<	Left shift
>>	Right shift

These operators are particularly useful in system-level and low-level programming.

8. Special Operators

C also has several operators that serve specialized purposes.

Examples include:

`sizeof`

Returns the size, in bytes, of a type or object.

```
sizeof(int)
```

Address Operator `&`

Obtains the address of an object.

```
&age
```

This is why introductory `scanf()` calls often use `&variable`.

Indirection Operator `*`

Used with pointers to access the object/value referred to by a pointer.

```
*ptr
```

Expressions in C

An **expression** is a combination of operands and operators that is evaluated to produce a value or, more generally, perform the computation specified by the expression.

Example:

```
a + b
```

Here, `a` and `b` are operands and `+` is the operator.

Another example:

```
a + b * c
```

is an arithmetic expression.

Definition: An expression is a combination of variables, constants, function calls, and operators that is evaluated according to C's rules.

Types of Expressions

1. Arithmetic Expression

```
a + b * c
```

2. Relational Expression

```
a > b
```

3. Logical Expression

```
a > b && b > c
```

4. Assignment Expression

```
a = b + 10
```

5. Conditional Expression

```
(a > b) ? a : b
```

Operator Precedence and Associativity

When an expression contains several operators, C follows rules of **precedence and associativity** to determine how the expression is grouped and evaluated.

Consider:

```
result = 10 + 5 * 2;
```

Multiplication has higher precedence than addition.

Therefore:

```
10 + (5 × 2)
= 10 + 10
= 20
```

It is **not**:

```
(10 + 5) × 2 = 30
```

Simplified Precedence Order

For basic exam questions, remember:

Parentheses → Unary operators → `*` `/` `%` → `+` `-` → Relational → Equality → Logical AND → Logical OR → Conditional → Assignment

Parentheses should be used when you want to make the intended grouping explicit.

OPERATOR	TYPE	ASSOCIAVITY
( ) [ ] . ->		left-to-right
++ -- + - ! ~ (*type*) * & sizeof	Unary Operator	right-to-left
* / %	Arithmetic Operator	left-to-right
+ -	Arithmetic Operator	left-to-right
<< >>	Shift Operator	left-to-right
< <= > >=	Relational Operator	left-to-right
== !=	Relational Operator	left-to-right
&	Bitwise AND Operator	left-to-right
^	Bitwise EX-OR Operator	left-to-right
|	Bitwise OR Operator	left-to-right
&&	Logical AND Operator	left-to-right
||	Logical OR Operator	left-to-right
? :	Ternary Conditional Operator	right-to-left
= += -= *= /= %= &= ^= |= <<= >>=	Assignment Operator	right-to-left
,	Comma	left-to-right